

MSC Project 1 (Time Series)

Introduction

The goal of this assignment is to design, implement, and evaluate a **time-series-enhanced predictive process monitoring pipeline**. You will work with at least one real-life (or realistically simulated) event log and one or more external or derived time series (for example, workload traces, resource utilization, system performance indicators, sensor measures, or demand forecasts). From a process-centric perspective, you will first build “classical” supervised process monitoring models using only event-log-based features (control-flow, temporal, case attributes). You will then extend these models by integrating time-series information that is aligned with case prefixes, and systematically investigate whether the additional data improves predictive performance and business usefulness. The concrete prediction task must be selected and justified by your group (e.g., outcome prediction, remaining time prediction, risk of SLA violation, prediction of long waiting times, prediction of resource overload). Emphasis is placed on sound preprocessing and synchronization between event data and time series, modular and testable software design, and rigorous empirical evaluation following best practices in supervised process mining.

Expected Deliverable (Implementation, Testing, and Prototype Software Documentation)

You are expected to deliver a **fully reproducible software artifact** in Python (preferably using PM4Py, scikit-learn, and optionally a deep learning library), along with concise documentation and tests.

The implementation must include:

1. A robust data ingestion and preprocessing layer for the event log (cleaning, filtering, case and activity selection) and for the time-series source(s) (resampling, smoothing, handling missing values).
2. A prefix generation and feature engineering module that constructs:
 - log-based features (e.g., control-flow encodings, aggregated case attributes, temporal features such as time since start, time of day, weekday), and
 - time-series-based features aligned to each prefix (e.g., sliding-window statistics, lagged values, or learned representations of the external time series at the time of the prefix).
3. At least one supervised prediction pipeline for a well-defined task (classification or regression), encapsulated as scikit-learn–style pipelines where possible (including preprocessing, model training, and prediction).
4. Baseline models using only event-log features and extended models that additionally rely on time-series features. The configuration of models and features must be controlled via a small number of clearly documented parameters (e.g., in a configuration file or at the top of the main script/notebook).

5. An evaluation script or notebook that runs all experiments end-to-end from raw data to final metrics and plots (no manual intermediate steps).

Your code must follow sound software engineering principles:

Use a public or university-hosted Git repository with a meaningful commit history and branches or issues for major features.

- Organize code into modules or packages (e.g., `data`, `features`, `models`, `evaluation`, `utils`) instead of a single monolithic script.
- Ensure that all non-trivial functions and classes contain docstrings that explain their purpose, inputs, and outputs.
- Provide a `requirements.txt` or environment file and clear installation instructions in a `README.md` so that the instructors can run your pipeline on their machines.

Testing: You must implement automated tests for core components of your system (e.g., log loading, prefix generation, feature extraction for a small synthetic example, simple model training and prediction). These tests should be runnable with a single command (for example, using `pytest`). The tests are not expected to cover every line, but should protect against typical data and encoding issues.

Prototype software documentation: Alongside the source code, you must provide:

- A short **technical documentation** (3–5 pages) that explains the architecture of your system, the data flow from raw log and time series to predictions, and the main design choices (encodings, models, hyperparameters). A simple architecture diagram is highly recommended.
- A **user-oriented usage guide** (which can be in the `README` or a separate document) that explains how to: (1) obtain or place the datasets, (2) configure the experiments (which prediction task, which models), and (3) run the full pipeline to reproduce the main results and figures.
- Optionally, a lightweight **prototype interface** (e.g., a Jupyter notebook “demo” or a minimal web or command-line interface) that demonstrates how an analyst could use your model for online monitoring of selected cases, including visualizations of predictions over time and the time-series context.

Expected Evaluation

Your evaluation must clearly answer the question: “**Does integrating time-series information improve predictive process monitoring, and under which conditions?**”

You are expected to design and conduct a rigorous experimental study, including:

1. A precise definition of the prediction task, target variable(s), and label generation strategy (e.g., case outcome, remaining time at the prefix, risk of SLA violation above a chosen threshold).
2. Construction of appropriate train/validation/test splits that respect the temporal nature of the data (for example, using time-based splits or rolling-origin evaluation to avoid unrealistic information leakage).
3. Training and tuning of at least one baseline model using only event-log features (for example, gradient boosting, random forests, or a suitable neural architecture) and at least one “enhanced” model that additionally exploits time-series-derived features.

Hyperparameters should be tuned in a principled way (e.g., grid or random search with a clear validation strategy).

4. Use of suitable evaluation metrics for the chosen task, such as AUC, accuracy, F1, Brier score, or calibration curves for classification; and MAE, RMSE, or MAPE for regression. When appropriate, you should additionally report prefix-length dependent performance (e.g., performance at early vs late stages of the case).
5. A comparative analysis of models with and without time-series signals, including statistical or at least qualitative significance assessments (for instance, confidence intervals, boxplots across multiple runs, or a careful discussion of observed differences).
6. Visualizations that help interpret the results, such as feature importance plots (if available), partial dependence plots for key time-series features, learning curves, or example trajectories of individual cases showing how predictions evolve over time as both the process and the external time series unfold.
7. A critical discussion section in your written report that reflects on:
 - the added value (or lack thereof) of time-series features in your specific scenario,
 - potential reasons for success or failure (e.g., low signal in the time series, misalignment, data quality issues),
 - computational and implementation overhead introduced by time-series integration, and
 - generalizability of your findings to other processes and time-series types.

The final written report should be approximately 10-15 pages (excluding references and appendices) and must combine technical detail (data preprocessing, model specification, evaluation protocol) with conceptual reflection (link to the supervised process mining literature, limitations, future work).

References

You are encouraged to consult and cite the following (non-exhaustive) list of references, and to add further relevant literature on time series and machine learning as needed:

1. van der Aalst, W.M.P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.
2. Teinemaa, I., Dumas, M., Rosa, M.L., & Maggi, F.M. (2019). Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Transactions on Knowledge Discovery from Data*, 13(2), 1–57.
3. Verenich, I., Dumas, M., Rosa, M.L., Maggi, F.M., & Teinemaa, I. (2019). Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring. *ACM Transactions on Intelligent Systems and Technology*, 10(4), 1–34.
4. Teinemaa, I., Dumas, M., Rosa, M.L., & Maggi, F.M. (2016). Predictive business process monitoring with structured and unstructured data. In *Business Process Management (BPM 2016)*, Springer.
5. Ramirez-Alcocer, U.M., et al. (2023). A deep learning approach for predictive healthcare process monitoring. *Information*, 14(9), 508.

6. Senderovich, A. (2017). Queue Mining: Service Perspectives in Process Mining. In *BPM (Demos)*.
7. Stevens, A., De Smedt, J., & Peeperkorn, J. (2021). Quantifying explainability in outcome-oriented predictive process monitoring. In *International Conference on Process Mining (ICPM)*.

You may also draw on standard textbooks or tutorials on time-series analysis and machine learning (for example, resources on feature-based time-series representations, sliding-window aggregation, or deep learning for sequence data), provided you cite them appropriately in your report.